



**Universidad Nacional
Autónoma de México**
Facultad de Ingeniería
División de ingeniería eléctrica



Computación gráfica avanzada

Semestre: 2026-2

Estudiante: Martínez Navarro Fernando David

Proyecto final: Manual técnico

Fecha de entrega:

13 de junio del 2026

índice

Manual técnico.....	3
1. Introducción.....	3
2. Objetivo del juego (Gameplay)	3
3. Diseño	3
3.1 Diseño visual y del escenario.....	3
3.2 Iluminación y ambiente	3
3.3 Interfaz y estados.....	4
3.4 Diseño de audio	4
4. Plan de trabajo	4
5. Licenciamiento de modelos y sonido.....	4
5.1 Efectos de sonido de Pixabay.....	5
5.2 Música	5
5.3 Modelos animados.....	5
6. Precio estimado.....	5
7. Desarrollo.....	6
7.1 Tecnologías y dependencias	6
7.2 Organización del proyecto	6
7.3 Renderizado y shaders	7
7.4 Modelos y animaciones	7
7.5 Colisiones y reglas.....	8
7.6 Iluminación, partículas y audio.....	8
7.7 Controles y cámaras	9
8. Resultados y trabajo a futuro.....	10
9. Conclusiones.....	10

Manual técnico

1. Introducción

El proyecto consiste en un videojuego tridimensional de exploración y supervivencia desarrollado en C++ con OpenGL. El jugador controla a un caballero que debe recorrer un laberinto, recolectar monedas y alcanzar la salida mientras evita enemigos y objetos trampa.

La aplicación integra renderizado 3D, modelos estáticos y animados, iluminación múltiple, sombras, partículas, colisiones, audio espacial, música dinámica, interfaz gráfica y soporte para teclado, mouse y control.

2. Objetivo del juego (Gameplay)

El objetivo principal es recolectar cinco monedas auténticas y llegar a la salida del laberinto. El jugador comienza con tres vidas, representadas mediante corazones en pantalla. Los enemigos reducen una vida al entrar en contacto con el jugador y se aplica un periodo breve de invulnerabilidad para evitar daño continuo.

Las cinco monedas reales incrementan el contador y emiten sonido.

Las cuatro monedas falsas provocan la derrota inmediata.

Al reunir cinco monedas se abren las puertas y se activa una luz en la salida.

La victoria se obtiene al cruzar la zona de salida con las puertas abiertas.

Al perder todas las vidas se reproduce la animación de muerte y aparece Game Over.

3. Diseño

El diseño del escenario y la ambientación se decidió con base en los videojuegos Dark Souls, con una temática oscura y lúgubre, de ahí que se tenga a un caballero como jugador principal y haya enemigos de tipo zombie dentro del laberinto, el cual cuenta con texturas que asemejan a mazmorras oscuras y muros de cueva o de ladrillos de murallas medievales.

3.1 Diseño visual y del escenario

El escenario se compone de un terreno con BlendMap y un laberinto modular. El canal rojo utiliza Sangre.jpg, el verde Camino.jpg, el azul Ceniza.jpg y el fondo SueloDark.jpg. NewBlendMap.png define la distribución de estas superficies. El cielo utiliza un Skybox y los muros, puertas, monedas y antorchas se cargan como modelos OBJ.

3.2 Iluminación y ambiente

La iluminación combina una luz direccional tenue, cinco PointLights naranjas asociadas a las antorchas y una PointLight de mayor intensidad que acompaña al jugador. Al abrirse las puertas se activa un Spotlight orientado hacia el suelo para señalar claramente la salida.

3.3 Interfaz y estados

La interfaz presenta un menú inicial, un contador de monedas, tres corazones para las vidas y pantallas independientes de victoria y derrota. La música cambia según el estado del juego y el número de monedas recogidas.

3.4 Diseño de audio

Las antorchas, monedas y enemigos utilizan fuentes de audio espaciales. Su reproducción depende de la distancia entre el jugador y cada objeto. Los sonidos de daño, apertura de puertas y música se reproducen como audio global cuando su función requiere que sean audibles desde cualquier punto.

4. Plan de trabajo

Etapa	Actividades principales	Horas invertidas
1. Preparación	Revisión del proyecto base, limpieza y organización.	10 horas
2. Escenario	Integración de laberinto, terreno, BlendMap y puertas.	15 horas
3. Gameplay	Monedas, vidas, trampas, victoria y derrota.	20 horas
4. Enemigos	Carga FBX, animaciones, patrullaje, OBB y daño.	4 horas
5. Ambiente	Antorchas, partículas, luces y sombras.	6 horas
6. Audio	Audio espacial, efectos y música por estados.	10 horas
7. Controles	Teclado, mouse, joystick y cámaras.	2 horas
8. Cierre	Eliminación de recursos de prueba y documentación.	2 horas

5. Licenciamiento de modelos y sonido

Tipo	Recursos integrados	Referencia
Modelos	Jugador y enemigos animados.	Mixamo; autores Paladin J Nordstrom y Warzombie F Pedroso. Diego G; Moneda de 1 peso.

Tipo	Recursos integrados	Referencia
Efectos	Fuego, monedas, daño, puerta y enemigos.	Creadores de Pixabay indicados en la lista de créditos.
Música	Temas de menú, batalla y Game Over.	EIM Studio, banda sonora de Library of Ruina.

5.1 Efectos de sonido de Pixabay

1. Sound Effect by [Vilches86](#) from [Pixabay](#).
2. Sound Effect by [Universfield](#) from [Pixabay](#).
3. Sound Effect by [freesound_community](#) from [Pixabay](#).
4. Sound Effect by [DRAGON-STUDIO](#) from [Pixabay](#).
5. Sound Effect by [freesound_community](#) from [Pixabay](#).
6. Sound Effect by [freesound_community](#) from [Pixabay](#).
7. Sound Effect by [Vilches86](#) from [Pixabay](#).

5.2 Música

- EIM Studio - Library of Ruina OST: Hokma battle theme 1.
- EIM Studio - Library of Ruina OST: Hokma battle theme 2.
- EIM Studio - Library of Ruina OST: Hokma battle theme 3.
- EIM Studio - Library of Ruina OST: Hokma battle Story.
- EIM Studio - Library of Ruina OST: 4_1_StoryChaboon.

5.3 Modelos animados

Mixamo: Paladin J Nordstrom.

Mixamo: Warzombie F Pedroso.

["1 Peso MXN - Moneda - Coin"](#) por Diego G. Licenciado bajo [Creative Commons Attribution 4.0 \(CC BY 4.0\)](#).

6. Precio estimado

Se propone un precio de lanzamiento de \$99 MXN para una versión académica publicable en computadora. Este precio considera que se trata de una experiencia breve de un solo nivel, con controles completos, ambientación, audio, enemigos, condición de victoria y derrota.

Para una versión comercial sería necesario ampliar el contenido, realizar pruebas en más equipos, verificar todas las licencias, añadir opciones de accesibilidad y preparar un instalador. Con esas mejoras el precio podría revisarse de acuerdo con la duración y calidad final.

7. Desarrollo

7.1 Tecnologías y dependencias

Tecnología	Uso dentro del proyecto
C++ / OpenGL 3.3	Lógica principal y renderizado gráfico.
GLFW	Ventana, teclado, mouse y control.
GLEW	Acceso a funciones modernas de OpenGL.
GLM	Vectores, matrices, transformaciones y cuaterniones.
Assimp	Carga de modelos OBJ y FBX con animaciones.
FreeImage	Carga de imágenes y texturas.
FreeType	Renderizado del contador de monedas.
OpenAL / ALUT	Audio global y espacial.
CGALib	Modelos, cámaras, terreno, shaders, colisiones y utilidades.
CMake / MinGW	Configuración y compilación del ejecutable.

Fragmento de implementación. 14-ProyectoFinal/CMakeLists.txt, líneas 7-18 y 33-42

```
include_directories(
    ${CMAKE_SOURCE_DIR}/14-ProyectoFinal/src
    ${CMAKE_SOURCE_DIR}/CGALib/include
    "../external/assimp/include"
    "../external/glfw/include"
    "../external/glm"
    "../external/glew/include"
    "../external/FreeImage/include"
    "../external/OpenAL/include"
    "../external/Freetype/include"
    "../external/freealut/include"
)

add_executable(14-ProyectoFinal ./src/main.cpp)
target_link_libraries(
    14-ProyectoFinal opengl32.lib libglfw3dll.dll.a
    libglew32.dll.a libassimp.dll.a libfreetype.dll.a
    libalut.dll.a OpenAL32.lib CGALib)
```

7.2 Organización del proyecto

El archivo 14-ProyectoFinal/src/main.cpp concentra la inicialización, estados de juego, controles, matrices de modelo, renderizado y ciclo principal. CGALib contiene las clases reutilizables. Las carpetas models, Textures, sounds, Fonts y Shaders almacenan los recursos cargados durante la ejecución.

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 644-690

```
modelEntrada.loadModel("../models/Laberinto/Entrada.obj");
modelEntrada.setShader(&shaderMulLighting);

modelMoneda1.loadModel("../models/Laberinto/Moneda1.obj");
modelMoneda1.setShader(&shaderMulLighting);

modelEnemy1.loadModel("../models/Enemy/Enemy1.fbx");
modelEnemy1.setShader(&shaderMulLighting);

playerModelAnimate.loadModel("../models/Player/Player.fbx");
playerModelAnimate.setShader(&shaderMulLighting);
```

7.3 Renderizado y shaders

La escena utiliza shaders separados para objetos iluminados, terreno, Skybox, texturas de interfaz, partículas y mapa de profundidad. El sistema de Shadow Mapping genera primero un Depth Map desde la dirección de la luz y después lo consulta durante el renderizado normal.

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 613-620

```
shaderSkybox.initialize(
    "../Shaders/skyBox.vs", "../Shaders/skyBox_fog.fs");
shaderMulLighting.initialize(
    "../Shaders/iluminacion_textura_animation_shadow.vs",
    "../Shaders/multipleLights_shadow.fs");
shaderTerrain.initialize(
    "../Shaders/terrain_shadow.vs", "../Shaders/terrain_shadow.fs");
shaderDepth.initialize(
    "../Shaders/shadow_mapping_depth.vs",
    "../Shaders/shadow_mapping_depth.fs");
shaderParticlesFire.initialize(
    "../Shaders/particlesFire.vs", "../Shaders/particlesFire.fs");
```

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 2357-2376

```
glViewport(0, 0, SHADOW_WIDTH, SHADOW_HEIGHT);
glBindFramebuffer(GL_FRAMEBUFFER, depthMapFBO);
glClear(GL_DEPTH_BUFFER_BIT);
glCullFace(GL_FRONT);
prepareDepthScene();
renderScene();
glCullFace(GL_BACK);
glBindFramebuffer(GL_FRAMEBUFFER, 0);

glActiveTexture(GL_TEXTURE10);
glBindTexture(GL_TEXTURE_2D, depthMap);
shaderMulLighting.setInt("shadowMap", 10);
shaderTerrain.setInt("shadowMap", 10);
renderSolidScene();
```

7.4 Modelos y animaciones

Los elementos del laberinto se manejan mediante matrices independientes. El jugador emplea animaciones de movimiento y muerte. Los cinco enemigos utilizan animación de caminata y giro, recorriendo distancias específicas entre dos extremos de su zona de patrullaje.

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 432-480

```
if (!patrol.turning) {
    model.setAnimationIndex(ENEMY_ANIMATION_WALK);
    float movement = glm::min(
        ENEMY_PATROL_SPEED * deltaSeconds,
        patrol.maxDistance - patrol.traveledDistance);
    patrol.traveledDistance += movement;
    if (patrol.traveledDistance >= patrol.maxDistance)
        patrol.turning = true;
} else {
    model.setAnimationIndex(ENEMY_ANIMATION_TURN);
    patrol.turnAngle += glm::min(
        ENEMY_TURN_SPEED * deltaSeconds,
        180.0f - patrol.turnAngle);
}

modelMatrix = glm::translate(baseMatrix, offset);
modelMatrix = glm::rotate(
    modelMatrix, glm::radians(orientation),
    glm::vec3(0.0f, 1.0f, 0.0f));
```

7.5 Colisiones y reglas

Las monedas y los muros emplean AABB por su bajo costo de cálculo. El jugador y los enemigos utilizan OBB para respetar mejor su orientación. Las puertas funcionan como obstáculos antes de reunir las monedas y como zonas de activación de victoria después de abrirse.

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 2380-2420

```
AbstractModel::OBB playerCollider;
glm::mat4 colliderMatrix = glm::mat4(modelMatrixPlayer);
colliderMatrix = glm::rotate(
    colliderMatrix, glm::radians(180.0f),
    glm::vec3(0, 1, 0));
playerCollider.u = glm::quat_cast(colliderMatrix);
playerCollider.e = playerModelAnimate.getOBB().e
    * glm::vec3(0.0085f, 0.015f, 0.0085f);

collidersEnemigosOBB["Enemy-1"] = createModelOBB(
    modelEnemy1, modelMatrixEnemy1, ENEMY_MODEL_SCALE);
collidersMonedasAABB["Moneda-1"] = transformAABB(
    modelMoneda1.getAABB(),
    glm::scale(modelMatrixMoneda1, glm::vec3(0.5f)));
```

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 2545-2611

```
if (testAABBAABB(playerAABB, it->second)) {
    monedasActivas[monedaIndex] = false;
    contadorMonedas++;
    alSourcePlay(source[COIN_PICKUP_SOURCE_INDEX]);
}

if (puertasAbiertas && !juegoGanado
    && (testAABBAABB(playerAABB, triggerPuertaIzq)
        || testAABBAABB(playerAABB, triggerPuertaDer))) {
    juegoGanado = true;
    std::cout << "Ganaste" << std::endl;
}
```

7.6 Iluminación, partículas y audio

Cada antorcha genera partículas de fuego y posee una luz puntual y una fuente de audio espacial. El jugador cuenta con iluminación móvil. Las monedas y enemigos actualizan sus fuentes según posición y distancia, mientras que la música cambia en el menú, al iniciar, con dos monedas, con cuatro monedas y al entrar en Game Over.

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 2159-2178 y 2229-2238

```
int exitSpotLightCount = puertasAbiertas ? 1 : 0;
shaderMulLighting.setInt(
    "spotLightCount", exitSpotLightCount);

if (puertasAbiertas) {
    glm::vec3 exitSpotPosition(
        exitCenter.x, 12.0f, exitCenter.z);
    glm::vec3 exitSpotDirection(0.0f, -1.0f, 0.0f);
}

const int playerLightIndex =
    static_cast<int>(antochaPositions.size());
shaderMulLighting.setInt(
    "pointLightCount", playerLightIndex + 1);
```

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 483-518

```
glGenBuffers(1, &fireInitVel);
glGenBuffers(1, &fireStartTime);

for (unsigned int i = 0; i < nParticlesFire; i++) {
    theta = glm::mix(
        0.0f, glm::pi<float>() / 10.0f,
        ((float)rand() / RAND_MAX));
    phi = glm::mix(
        0.0f, glm::two_pi<float>(),
        ((float)rand() / RAND_MAX));
    v.x = sinf(theta) * cosf(phi);
    v.y = cosf(theta);
    v.z = sinf(theta) * sinf(phi);
}
```

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 1022-1038 y 2713-2729

```
buffer[0] = alutCreateBufferFromFile(
    "../sounds/FuegoMono.wav");
buffer[1] = alutCreateBufferFromFile(
    "../sounds/BrillanteMono.wav");
buffer[ENEMY_AUDIO_BUFFER_INDEX] =
    alutCreateBufferFromFile("../sounds/ZombieMono.wav");

const bool playerInTorchAudioRange =
    glm::distance(
        playerAudioPosition, torchAudioPosition) <= 18.0f;
if (playerInTorchAudioRange && sourceState != AL_PLAYING)
    alSourcePlay(source[sourceIndex]);
else if (!playerInTorchAudioRange
    && sourceState == AL_PLAYING)
    alSourcePause(source[sourceIndex]);
```

7.7 Controles y cámaras

El modo principal utiliza una cámara en tercera persona. También existen cámaras libre y aérea para inspección. El joystick usa zona muerta para evitar movimientos involuntarios; el stick izquierdo controla al personaje y el derecho la cámara, sin utilizar los gatillos para la inclinación.

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 1305-1320 y 1381-1429

```
const bool gamepadConnected =
    glfwJoystickIsGamepad(GLFW_JOYSTICK_1) == GLFW_TRUE
    && glfwGetGamepadState(
        GLFW_JOYSTICK_1, &gamepadState) == GLFW_TRUE;

const float leftX =
    gamepadState.axes[GLFW_GAMEPAD_AXIS_LEFT_X];
const float leftY =
    gamepadState.axes[GLFW_GAMEPAD_AXIS_LEFT_Y];
const float rightX =
    gamepadState.axes[GLFW_GAMEPAD_AXIS_RIGHT_X];
const float rightY =
    gamepadState.axes[GLFW_GAMEPAD_AXIS_RIGHT_Y];

camera->mouseMoveCamera(cameraX, cameraY, deltaTime);
```

Fragmento de implementación. 14-ProyectoFinal/src/main.cpp, líneas 1470-1483

```
if (glfwGetKey(window, GLFW_KEY_LEFT) == GLFW_PRESS)
    modelMatrixPlayer = glm::rotate(
        modelMatrixPlayer, 0.05f, glm::vec3(0, 1, 0));

if (glfwGetKey(window, GLFW_KEY_UP) == GLFW_PRESS) {
    modelMatrixPlayer = glm::translate(
        modelMatrixPlayer, glm::vec3(0.0, 0.0, 0.5));
    animationPlayerIndex = 1;
}
```

8. Resultados y trabajo a futuro

El resultado es una experiencia jugable completa: menú, exploración, coleccionables, enemigos animados, vidas, trampas, puertas, victoria, derrota, HUD, iluminación, partículas y audio dinámico. El proyecto se compila como un único objetivo junto con CGALib.

Añadir más niveles y variaciones del laberinto.

Incorporar animaciones de ataque y comportamientos de persecución.

Agregar pausa, reinicio, configuración de volumen y sensibilidad.

Crear puntos de guardado, cronómetro y sistema de puntuación.

Optimizar la carga de recursos y separar main.cpp en módulos.

Completar atribuciones y licencias antes de una distribución pública.

Realizar pruebas de rendimiento, accesibilidad y compatibilidad.

9. Conclusiones

El proyecto permitió integrar de forma práctica los principales temas del curso de Computación Gráfica Avanzada. La combinación de renderizado, animación, colisiones, iluminación, partículas, audio e interacción produjo un videojuego funcional con una identidad visual y sonora coherente.

La implementación demuestra que una arquitectura basada en OpenGL y una biblioteca auxiliar puede sostener un ciclo de juego completo, además de que con algo más de inversión en el uso de esa arquitectura, se pueden hacer proyectos más ambiciosos como el desarrollo de un motor gráfico gracias a la gran capacidad de OpenGL para usar los

recursos de la computadora y hacer manipulación de estos elementos en tiempo de ejecución.